

## Brain Ct Triage Challenge

### Brain CT Triage Challenge

#### Overview

In this challenge, you will develop models that perform clinically meaningful triage on non-contrast brain CT scans.

Instead of directly predicting triage classes, your model will predict a set of intermediate imaging quantities for each CT study:

- Total volume (in mL) of each intracranial hemorrhage (ICH) subtype.
- Probability of any skull fracture.
- Midline shift (MLS) in mm.

A fixed, clinically motivated rule (provided as executable Python code) then converts these intermediate values into triage classes:

- **0** – Non-urgent
- **1** – Urgent
- **2** – Critical

The primary evaluation metric is **Macro F1 Score (f1-macro)** between your model's triage output and the ground-truth triage.

Participants will not submit CSV prediction files directly. Instead, you will submit:

1. Your model files (weights, configs, etc.), and
2. A Python module implementing a standard Model API.

Our evaluation pipeline will load your code and models and run inference directly on the hidden evaluation set.

---

#### Problem Description

Non-contrast brain CT is a first-line imaging modality in emergency neuroradiology. Rapid triage is crucial to identify patients who:

- Need immediate neurosurgical or neurocritical care (**critical**),
- Require urgent but not emergent attention (**urgent**),

- Have no acute, triage-relevant findings (**non-urgent**).

Clinically, triage is primarily driven by a small set of imaging primitives:

- Intracranial hemorrhage burden (how much blood, and of what type?),
- Presence of skull fracture,
- Degree of midline shift / mass effect.

You will build models that predict these primitives from CT images. A transparent, fixed triage rule converts those primitives to triage classes. This design ensures:

- **Interpretability** – your model predicts clinically meaningful quantities.
- **Mechanistic behavior** – triage is derived from explicit rules.
- **Fairness / reproducibility** – the same rule is used for both ground truth and predictions.

---

## Labels and Tasks

### Available Annotations (Training Set)

For each CT study in the training set, we provide:

1. **Intracranial hemorrhage (ICH) segmentation masks:** Five semantic classes (exact names defined in the Data Profile), typically including:
  - Epidural hemorrhage (EDH)
  - Subdural hemorrhage (SDH)
  - Intraparenchymal / intracerebral hemorrhage (IPH/ICH)
  - Subarachnoid hemorrhage (SAH)
  - Intraventricular hemorrhage (IVH)

*There may be multiple lesions per ICH subtype. All instances of a given subtype are included in its mask. The volume per subtype is defined as the combined volume of all lesions of that subtype in the study.*

2. **Skull fractures:** 2D bounding boxes indicating the locations of skull fractures on each annotated slice.

3. **Midline shift keypoints:** For slices where midline annotations are provided, three keypoints are stored:

- "AnteriorFalxAttachment"
- "PosteriorFalxAttachment"
- "OutermostPointOfTheFalx"

*These keypoints can be used to reconstruct the midline and compute slice-wise midline deviation. For the challenge, ground-truth midline shift at the foramen of Monro level (*

*MLS\_mm*

*) is computed from these keypoints and provided as part of the intermediate ground truth for each series.*

These annotations are provided for training (and optionally a labeled validation subset). The test set includes only the CT images (no segmentation, no keypoints, no fracture boxes).

---

## Data Profile & Dataset Structure

The dataset consists of three main components:

### 1. Metadata

A metadata file is provided (e.g.,

metadata.csv

or

metadata.parquet

) that contains:

- DICOM-level information (e.g.,

dicom\_series.id

, number of slices, acquisition parameters, etc.).

- Aggregated labels derived from per-slice annotations and keypoints, including:

- Series-level ICH volumes per subtype:

V\_EDH

,

V\_SDH

,

V\_IPH

,

V\_SAH

,

V\_IVH

(mL)

- Series-level midline shift:

MLS\_mm

(mm at the foramen of Monro level)

- Series-level fracture presence:

SkullFracture

(boolean) or fracture count, which maps to

fracture\_prob = 0

or

1

in the intermediate schema.

- Optional triage labels for the training set (for debugging).

*This metadata is primarily for training and quality control; the official triage labels are always derived via*

*triage\_from\_intermediates*

*from the intermediate primitives.*

## **2. Raw DICOM Data**

All raw images are provided as DICOM files under a root directory:

dicom/

└─ {dicom\_series.id}/

└─ \*.dcm

└─ ...

For each series,

dicom/{dicom\_series.id}/

contains all DICOM slices belonging to that series. Your model's

`predict(study_dir)`

method will receive the path:

`study_dir = "dicom/{dicom_series.id}"`

. You are free to use any DICOM loading strategy that works inside the official environment (e.g.,

`pydicom`

,

`SimpleITK`

,

`MONAI`

loaders).

### 3. Annotation JSON Files

Per-slice annotations (segmentation, keypoints, fracture boxes) are stored as JSON files:

annotations/

└─ {dicom\_series.id}/

└─ {SOPInstanceUID}.json

Each JSON file for a given DICOM instance (

`SOPInstanceUID`

) contains:

- A segmentation map encoded as a 2D label mask (ICH subtypes encoded as integer labels) stored as RLE (Run-Length Encoding).
- A class map describing the mapping from integer label → class name.
- A keypoints dictionary mapping keypoint name →

(x, y)

coordinates (or

None

).

- A list of boxes in

[x, y, w, h]

format, typically used for skull fractures.

---

## Intermediate Quantities & Triage Rules

### Intermediate Schema (per series)

For each CT series, your model must output the following intermediate values:

- "V\_EDH"

– total epidural hemorrhage volume in mL

- "V\_SDH"

– total subdural hemorrhage volume in mL

- "V\_IPH"

– total intraparenchymal / intracerebral hemorrhage volume in mL

- "V\_SAH"

– total subarachnoid hemorrhage volume in mL

- "V\_IVH"

– total intraventricular hemorrhage volume in mL

- "fracture\_prob"

– probability (0–1) of any skull fracture in the series

- "MLS\_mm"

– midline shift at the foramen of Monro level in mm

The same schema is used for both ground truth intermediates (

y\_true[series\_id]

) and predicted intermediates (

y\_pred[series\_id]

).

### **Official Triage Function**

This is the canonical triage function that will be used to derive triage labels from intermediate values:

from typing import Any, Dict, Mapping

```
TRIAGE_REQUIRED_KEYS = {
```

```
    "V_EDH", "V_SDH", "V_IPH", "V_SAH", "V_IVH", "fracture_prob", "MLS_mm",
```

```
}
```

```
def validate_intermediates(intermediates: Mapping[str, Any]) -> Dict[str, float]:
```

```
    """Validate and normalize intermediate values for a single series."""
```

```
    missing = TRIAGE_REQUIRED_KEYS - intermediates.keys()
```

```
    extra = intermediates.keys() - TRIAGE_REQUIRED_KEYS
```

```
    if missing:
```

```
        raise ValueError(f"Missing keys in intermediates: {sorted(missing)}. Expected: {sorted(TRIAGE_REQUIRED_KEYS)}.")
```

```
    if extra:
```

```
    raise ValueError(f"Unexpected keys in intermediates: {sorted(extra)}. Expected: {sorted(TRIAGE_REQUIRED_KEYS)}.")
```

```
    cleaned: Dict[str, float] = {}
```

```
    for key in TRIAGE_REQUIRED_KEYS:
```

```
        try:
```

```
            cleaned[key] = float(intermediates[key])
```

```
        except Exception as exc:
```

```
            raise TypeError(f"Value for key {key!r} must be convertible to float, got type {type(intermediates[key]).__name__}.") from exc
```

```
    return cleaned
```

```
def triage_from_intermediates(intermediates: Mapping[str, Any]) -> int:
```

```
    """Compute triage class from intermediate imaging primitives."""
```

```
    vals = validate_intermediates(intermediates)
```

```
    # Extract and clamp primitives
```

```
    V_EDH = max(0.0, vals["V_EDH"])
```

```
    V_SDH = max(0.0, vals["V_SDH"])
```

```
    V_IPH = max(0.0, vals["V_IPH"])
```

```
    V_SAH = max(0.0, vals["V_SAH"])
```

```
    V_IVH = max(0.0, vals["V_IVH"])
```

```
    MLS_mm = max(0.0, vals["MLS_mm"])
```

```
    fracture_prob = float(vals["fracture_prob"])
```

```
    total_vol = V_EDH + V_SDH + V_IPH + V_SAH + V_IVH
```

```
    # Hard-coded thresholds
```



EPS\_VOLUME = 0.1    *# mL: ignore tiny volumes as noise*  
EPS\_MLS = 1.0      *# mm: ignore <1 mm as no meaningful shift*  
MLS\_CRITICAL = 5.0    *# mm: high MLS threshold*  
MLS\_URGENT\_LOW = 3.0    *# mm: moderate MLS range lower bound*  
EDH\_CRIT = 30.0    *# mL*  
SDH\_CRIT = 70.0    *# mL*  
IPH\_CRIT = 70.0    *# mL*  
TOTAL\_VOL\_CRIT = 60.0 *# mL*  
COMBO\_MLS = 3.0    *# mm: for MLS + volume combo rule*  
COMBO\_VOL = 40.0    *# mL*  
FRAC\_VOL\_CRIT = 15.0 *# mL: fracture + hemorrhage for critical*  
FRACTURE\_PRESENCE\_THRESHOLD = 0.5 *# fracture probability threshold*

*# Derived flags*

has\_ich = total\_vol >= EPS\_VOLUME

mls\_present = MLS\_mm >= EPS\_MLS

fracture\_present = fracture\_prob >= FRACTURE\_PRESENCE\_THRESHOLD

*# -----*

*# Critical triage (2)*

*# -----*

if MLS\_mm >= MLS\_CRITICAL and (has\_ich or fracture\_present): return 2

if V\_EDH >= EDH\_CRIT: return 2

if V\_SDH >= SDH\_CRIT: return 2

if V\_IPH >= IPH\_CRIT: return 2

if total\_vol >= TOTAL\_VOL\_CRIT: return 2

```

if has_ich and MLS_mm >= COMBO_MLS and total_vol >= COMBO_VOL: return 2

if fracture_present and total_vol >= FRAC_VOL_CRIT: return 2


# -----
# Urgent triage (1)
# -----

if MLS_mm >= MLS_CRITICAL and not (has_ich or fracture_present): return 1

if has_ich: return 1

if MLS_URGENT_LOW <= MLS_mm < MLS_CRITICAL: return 1

if fracture_present and total_vol < FRAC_VOL_CRIT: return 1

if total_vol >= EPS_VOLUME and mls_present: return 1


# -----
# Non-urgent triage (0)
# -----

return 0

```

---

## Evaluation Pipeline Sample

While the final competition submission requires model files and a Python module, the evaluation pipeline internally generates or expects predictions in the following format for validation and testing.

### Data Structure

You are given **68 brain CT series**, each stored in its own directory. Every directory represents one complete CT series made up of multiple DICOM slices.

```

.
├── series_0
│   ├── slice0.dcm

```

```

|   ├── ...
|   └── slicen.dcm
|
|   ├── series_1
|   |
|   |   ├── file0.dcm
|   |   |
|   |   ├── ...
|   |   |
|   |   └── filen.dcm
|   |
|   └── ...
|
└── series_n

```

Each directory contains all slices required to reconstruct and analyze that series.

### Submission Format

After implementing your solution in

submission.py

, generate predictions by running:

```
python submission.py --data-dir /path/to/data-dir --predictions-file-path
/path/to/submission.csv
```

Once the script finishes, it should create a

submission.csv

file.

### CSV Example

```
series_id,V_EDH,V_SDH,V_IPH,V_SAH,V_IVH,fracture_prob,MLS_mm
```

```
series_0,0.11,0.9,0.92,0.71,0.67,0.23,2.15
```

```
series_1,0.13,0.3,0.31,0.7,0.67,0.01,2.3
```

```
...
```

```
series_n,0.41,0.2,0.89,0.51,0.67,0.15,0.26
```

### Same Example as a Table

| series_id | V_EDH | V_SDH | V_IPH | V_SAH | V_IVH | fracture_prob | MLS_mm |
|-----------|-------|-------|-------|-------|-------|---------------|--------|
| series_0  | 0.11  | 0.90  | 0.92  | 0.71  | 0.67  | 0.23          | 2.15   |
| series_1  | 0.13  | 0.30  | 0.31  | 0.70  | 0.67  | 0.01          | 2.30   |
| ...       | ...   | ...   | ...   | ...   | ...   | ...           | ...    |
| series_n  | 0.41  | 0.20  | 0.89  | 0.51  | 0.67  | 0.15          | 0.26   |

---

## Model API & Environment

### Environment Requirements

- **Make sure you have Python v3.12 installed!**
- **CUDA Version 12.8**
- For exact dependency and environment details, refer to the official configuration:  
 [pyproject.toml](#)

### Evaluation Environment

All models will be evaluated in a controlled environment with:

- **Hardware:** 1 × GPU with 24 GB VRAM (e.g., NVIDIA RTX-class GPU). Sufficient CPU and RAM for typical medical imaging workloads.
- **Software:** A predefined Docker image containing Python 3.12, PyTorch, TensorFlow/Keras, commonly used libraries,

pydicom

, and supporting packages.

### Important:

- Your code must run in this environment without installing extra packages at evaluation time.
  - Your model must complete inference on the hidden evaluation set within a global time limit on a single 24 GB GPU (target  $\leq 30$  minutes; design for  $\leq 15$  minutes).
  - All prediction logic, including preprocessing, batching, and ensembling, must be implemented inside your code.
-

## Rules

1. **Data Usage:** Only use the official competition data and any external resources explicitly allowed by the organizers. Do not attempt to re-identify patients; all data are anonymized.
2. **Annotations:** You may fully use the provided segmentation masks, fracture boxes, and keypoints for training. You must not manually label or otherwise access ground truth on the hidden test set.
3. **Model Submission:** Your submission must contain

model.py

and a

models/

directory with all necessary weights. Your model must run in the official Docker image and satisfy the time limit on a single 24 GB GPU.

4. **Ensembling:** Ensembling multiple models is allowed, as long as the full inference pipeline completes within time and memory limits.
5. **Reproducibility:** Top teams may be required to share additional details (training code, configs, etc.) to verify results.
6. **Fair Play:** Sharing of general ideas and open-source code is encouraged. Sharing of private test predictions, test labels, or attempts to reverse-engineer the evaluation system is strictly prohibited. Organizers reserve the right to disqualify teams for non-compliance or suspicious behavior.